EN.601.447/647 Computational Genomics: Sequences

Homework 4 (Fall 2026)

Out of 31 points (20 written, 11 programming).

Programming questions

1. **Approximate suffix-prefix overlap**. (3 pts)

   Submit this question as `hw4q1`*, with the appropriate suffix:* `.py`*,* `.java` *or* `.cpp`*.*

   When we assemble a genome from reads, we look for pairs of reads where the end of one overlaps the start of the other. Because of sequencing errors, the overlap may not be exact.

   Write a program that reads two strings X and Y and finds the best overlap between a suffix of X and a prefix of Y. Score an alignment of a suffix of X with a prefix of Y by adding $+1$ for each position where the letters match and -2 for each mismatch, insertion or deletion. Give each prefix of Y the score of its best alignment with any suffix of X. The best overlap is the prefix with the highest score; if several prefix lengths tie, take the longest. The suffix and the prefix may be empty, and aligning the empty suffix with the empty prefix scores 0, so the best score is never negative.

   The input file has X on its first line and Y on its second. Each has between 1 and 500 characters, all `A`, `C`, `G` or `T`. Your program will output two numbers on one line: the length of the best prefix of Y, and its score. The program will be run like this:

   ```
   python3 hw4q1.py input.txt output.txt
   ```

   You will need to think about how to initialize the first row and the first column, given that the suffix of X may begin anywhere in X but the prefix of Y must begin at $Y[0]$, and about which cells describe alignments that reach the end of X.

   Example. If `input.txt` contains

   ```
   GGATCCATGCAT
   CATGGATCTA
   ```

   then `output.txt` should contain

   ```
   7 4
   ```

   The last three letters of X, `CAT`, match the first three of Y exactly, for a score of 3. But the prefix `CATGGAT` of Y aligns with the suffix `CATGCAT` of X with 6 matches and 1 mismatch, for a score of $6 - 2 = 4$, which is better. This example is given as a visible Gradescope test.

2. **Counting optimal alignments**. (4 pts)

   Submit this question as `hw4q2`*, with the appropriate suffix:* `.py`*,* `.java` *or* `.cpp`*.*

Two strings can have more than one alignment that "tie" for the best alignment score.

Write a program that reads two strings X and Y and writes the edit distance between them, where a mismatch, an insertion and a deletion each cost 1, followed by the number of different alignments of X and Y that achieve that distance. Two alignments are different if, written out as two gapped rows, they differ in at least one column (example below).

The input file has X on its first line and Y on its second. Each has between 1 and 60 characters, all `A`, `C`, `G` or `T`. For every test, the number of optimal alignments fits in a signed 64-bit integer (`long` in Java, `long long` in C++). Write the two output numbers (the edit distance and the number of optimal alignments) on one line. The program will be run like this:

```
python3 hw4q2.py input.txt output.txt
```

Example. If `input.txt` contains

```
TAGGCAT
TAGCAAT
```

then `output.txt` should contain

```
2 5
```

The edit distance is 2, and these are the five alignments that achieve it:

```
TAGGCAT    TAGGC-AT    TAGGC-AT    TAGGCA-T    TAGGCA-T
TAGCAAT    TA-GCAAT    TAG-CAAT    TA-GCAAT    TAG-CAAT
```

This example is given as a visible Gradescope test.

Important: some tests have *billions* or *trillions* of alignments tied for best. A program that identifies each of them one at a time will run out of time. Consider a strategy that builds a second matrix alongside the edit distance matrix, in which each cell holds the number of optimal alignments between the corresponding prefixes of X and Y.

Second example. If `input.txt` contains 40 `A`s and then 20 `A`s

```
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAA
```

then `output.txt` should contain

```
20 137846528820
```

Any 20 of the 40 `A`s in X can be the ones deleted, so there are $\binom{40}{20} = 137{,}846{,}528{,}820$ optimal alignments. Listing them one at a time would take far longer than the time limit, even in C++. This example is also a visible Gradescope test.

3. **Global alignment with affine gaps**. (4 pts)

Submit this question as `hw4q3`*, with the appropriate suffix:* `.py`*,* `.java` *or* `.cpp`*.*

In lecture, every gap column cost the same, so a gap of length L cost L times as much as a gap of length 1. But when DNA is being replicated, one insertion or deletion event will tend to add or remove several letters at once, creating a single long gap. An *affine* gap penalty involves two kinds of costs; a gap "open" cost (for "opening" a new gap), plus an "extension" cost (for "extending" the gap by one more letter). This was not covered in lecture, but you can read about it in Gusfield Section 11.8.6 or Jones and Pevzner Section 6.9.

Write a program that reads two strings X and Y and finds the lowest cost of a global alignment of X and Y, where a match costs 0, a mismatch costs 3, and a gap of length L costs $4 + L$. A *gap* is a maximal run of consecutive columns that each have a letter of X over a gap symbol, or a maximal run that each have a gap symbol over a letter of Y. Write three numbers on one line: the lowest cost of an alignment whose last column

(i) pairs the last letter of X with the last letter of Y,
(ii) has the last letter of X over a gap, and
(iii) has a gap over the last letter of Y.

The lowest cost of any alignment is the smallest of the three.

The input file has X on its first line and Y on its second. Each has between 1 and 500 characters, all `A`, `C`, `G` or `T`. The program will be run like this:

```
python3 hw4q3.py input.txt output.txt
```

One edit distance matrix is not enough here. Whether a gap column costs 1 or 5 depends on whether the column before it is part of the same gap, and the best alignment of two prefixes does not always end the way a longer alignment needs it to. Keep one matrix for each of the three kinds of last column, as in the readings.

Example. If `input.txt` contains

```
AAAAGCTC
AACTC
```

then `output.txt` should contain

```
7 16 15
```

These are the best alignments of each kind:

```
(i)   AAAAGCTC     (ii)  AAAAGCTC     (iii) AAAAGCTC--
      AA---CTC           AACTC---           AA-----CTC
```

Alignment (i) has one gap of length 3, costing $4 + 3 = 7$, and no mismatches. Alignment (ii) has three mismatches and a gap of length 3, for $9 + 7 = 16$. Alignment (iii) has gaps of length 5 and 2, for $9 + 6 = 15$. With a cost of 5 for every gap column and no opening cost, alignment (i) would cost 15. This example is given as a visible Gradescope test.

Written questions

1. **Global alignment by hand**. (4 pts)

   Let X = `GGCACT` label the rows and Y = `AGTCA` label the columns of a global alignment matrix. Use the cost function from lecture: 0 for a match, 2 for a transition (a mismatch between `A` and `G`, or between `C` and `T`), 4 for a transversion (any other mismatch) and 8 for a gap. Lower costs are better.

   (a) (3 pts) Fill in the whole matrix, and give the cost of an optimal global alignment of X and Y.

   (b) (1 pt) There is exactly one optimal alignment. Circle the cells your traceback passes through, and write the alignment out as two rows, with `-` for a gap.

2. **Local alignment by hand**. (3 pts)

   Let X = `CAAGGCCT` label the rows and Y = `AAGTCCT` label the columns of a local alignment matrix. Use the scores from lecture: $+2$ for a match, -4 for a mismatch and -6 for a gap. Higher scores are better.

   (a) (2 pts) Fill in the whole matrix.

   (b) (1 pt) Give the score of the best local alignment, circle the cells its traceback passes through, and write out the substrings of X and Y that it aligns.

3. **Approximate matching by hand**. (3 pts)

   Let P = `GCGT` label the rows and T = `GCGAGTTCCG` label the columns of the matrix from lecture for finding approximate occurrences of P in T: an edit distance matrix whose first row is all zeros. Remember that offsets are 0-based.

   (a) (2 pts) Fill in the whole matrix.

   (b) (1 pt) An occurrence of P with at most 1 edit ends at offset j if its last letter is $T[j]$. List every offset in T where such an occurrence ends. For each one, give the substring of T that P matches there, found by tracing back.

4. **Linear versus affine gaps**. (3 pts)

   Let $X =$ `GATCTGCAA` and $Y =$ `GTCGAA`. Here are two alignments of them:

   ```
   (1)          (2)
   GATCTGCAA    GATCTGCAA
   G---TCGAA    G-TC-G-AA
   ```

   A mismatch costs 3 throughout.

   (a) (1 pt) With a linear gap penalty, where every gap column costs 5, what does each alignment cost?

   (b) (1 pt) With the affine gap penalty of Programming Question 3, where a gap of length L costs $4 + L$, what does each alignment cost?

   (c) (1 pt) Keep the extension cost at 1 for each gap column, but let the opening cost be o, so a gap of length L costs $o + L$. For which values of o does alignment (1) cost less than alignment (2)? In one sentence, why might a biologist prefer alignment (1)? The readings for Programming Question 3 discuss this.

5. **Greedy shortest common superstring**. (4 pts)

   Run the greedy shortest common superstring algorithm from lecture on these five reads, with a minimum overlap length of 1:

   `CGATTCA  CATCCGA  CCGCCGA  TCCGAGG  CCGAGGT`

   At each step, greedy finds the ordered pair of strings (A, B) with the longest overlap between a suffix of A and a prefix of B, and replaces A and B with their merge. The strings it compares include the merges it has already made. There are no ties at any step. For each of the four merges, give A, B, the length of their overlap and the merged string. Then give the final superstring and its length.

6. **When greedy goes wrong**. (3 pts)

   Run the same greedy algorithm, again with a minimum overlap length of 1, on these three reads:

   `GTGGT  TGGAT  GGTGG`

   There are no ties at any step.

   (a) (1 pt) Give each merge as before, and the final superstring and its length.

   (b) (1 pt) Give a common superstring of the three reads that is shorter than greedy's.

   (c) (1 pt) Which of greedy's merges was the mistake, and why did taking the longest overlap first make the final superstring longer?

7. **Special review problem: inferring costs.** (0 pts)

Ungraded. A similar problem will be on Midterm 2, worth about 4 points. Solutions will be posted.

Below is a global alignment matrix for $X =$ `GGTAG` (rows) and $Y =$ `TAGAC` (columns), filled in as in lecture with a cost function of this form: 0 for a match, one cost for a transition, another for a transversion (as defined in Written Question 1), and another for a gap. All three costs are positive whole numbers. The matrix's first row and first column are hidden for an added challenge.

	ϵ	T	A	G	A	C
ϵ						
G		4	4	6	9	12
G		7	5	4	7	10
T		6	8	7	8	8
A		9	6	9	7	10
G		12	9	6	9	11

(a) What is the gap cost? Briefly explain your reasoning.

(b) What are the transition and transversion costs? Briefly explain your reasoning.

(c) There is exactly one optimal alignment. Write it out as two rows, with `-` for a gap. Report the number of edit operations of each type (M, S, D, I) in the optimal alignment.